Computers, Algorithms, and Boolean Logic

Algorithms, Programming, and Data Representation · Module 92313 · Chapter 2

From Fundamentals to AI Apocalypse · personal study guide

Contents

Start from memory 2

How to use this guide 2

How the chapter fits together 3

2.1 Algorithms are finite, executable procedures 4

2.2 A stored-program computer executes instructions 5

2.3 Programs have syntax, static meaning, and behavior 6

2.4 Propositions turn logic into two-valued inputs 7

2.5 Boolean algebra and truth tables make the rules checkable 8

2.6 Boolean laws simplify expressions 10

2.7 Logic gates implement Boolean functions 12

2.8 Physical circuits take time to switch 13

Formula map and symbols 13

Derive it cold 14

Explain it 14

Mixed practice 15

Exam-style problems 16

Self-check 17

Answers 18

This lecture connects a recipe for solving a problem to a machine that executes instructions, then shows how Boolean logic can be represented as algebra and circuits. The numbered sections below group the slides by topic because the supplied deck has slide numbers but no numbered topic sections.

Start from memory

Try these before reading. Write a concrete answer first; a blank or uncertain answer is useful feedback.

1. What makes a procedure an algorithm rather than an informal idea?
2. What is the difference between syntax and semantics?
3. How many input rows are needed for a truth table with three variables?
4. When is A $\Rightarrow$ B false?
5. What does it mean for NAND to be functionally complete?

Answers: see "Start from memory" under Answers at the end.

How to use this guide

Read one short topic, hide it, and reconstruct its definition, rule, or diagram from memory. Attempt the mixed problems before opening their worked answers.

1. **Day 0:** Do the cold start, study §§2.1–2.3, and explain the algorithm and stored-program ideas without looking.
2. **Day 1:** Rebuild the propositional logic rules in §2.4 and solve mixed practice 1–8.
3. **Day 3:** Work §§2.5–2.6 from memory; solve practice 9–15 and check every truth-table row.
4. **About day 7:** Reconstruct the gate circuits in §2.7, then solve practice 16–20 in mixed order.
5. **About day 14:** Re-derive the NAND circuit and take one exam-style problem under a timer.
6. **Before the exam:** Complete the self-check and both exam problems. Revisit each section named in a missed answer.

When stuck: attempt one small input by hand, name the rule you are applying, then check the result against a table or the definitions. If the prerequisites are missing, study one worked example and solve a fresh one cold.

How the chapter fits together

The route begins with precise procedures and machine execution, then moves from statements to Boolean expressions and finally to gates that implement those expressions.

Measured · describe what happens

- **Algorithm** §2.1

  input → finite steps → output

  Rests on: instructions that are defined, executable, and able to halt.

- **Stored-program machine** §2.2

  fetch · execute · test · stop

  Rests on: instructions stored in memory and executed by the machine.

- **Program language** §2.3

  syntax · static semantics · meaning

  Rests on: legal expressions that a compiler or interpreter can evaluate.

Defined · set the rules

- **Propositions** §2.4

  true = 1 · false = 0

  Rests on: statements with exactly one truth value.

- **Boolean expressions** §2.5

  $\neg \cdot \wedge \cdot \vee$

  Rests on: defined operations over the two truth values.

- **Truth tables** §2.5

  n variables → 2^n rows

  Rests on: checking every possible input assignment.

Derived · simplify and build

- **Boolean laws** §2.6

  (A + B)' = A' · B'

  Rests on: equal outputs for every assignment.

- **Logic gates** §2.7

  NAND alone is complete

  Rests on: gate outputs implementing Boolean functions.

- **Physical circuits** §2.8

  switching takes time

  Rests on: real components that need time to change state.

Question	Tool
Does a procedure meet the algorithm definition?	Check definition, executability, finite resources, and halting.
What does a Boolean expression produce?	Evaluate it row by row in a truth table.
Can a circuit use only NAND gates?	Rewrite NOT, AND, and OR using NAND and Boolean laws.

2.1 Algorithms are finite, executable procedures

An algorithm turns a problem into steps that a person or machine can carry out. A precise procedure also makes it possible to decide whether it finishes and what output it should produce.

From declarative facts to imperative steps

Declarative knowledge states facts. **Imperative knowledge** describes how to do something: a sequence of actions with a flow of control and a stopping condition. An algorithm is a finite, well-defined sequence of executable instructions for a class of problems or a computation.

Algorithm checklist

$$\text{input} \rightarrow \text{defined instructions} \rightarrow \text{output}$$

$$\text{finite text} \cdot \text{executable steps} \cdot \text{finite memory} \cdot \text{finite steps}$$

The slides call finite memory use dynamic finiteness and a finite number of steps halting. A procedure that never stops does not deliver its required result.

An algorithm is more specific than "find a square root": it says how to make and check successive guesses. The procedure in the notes starts with a guess g; if g^2 is close enough to x, it stops. Otherwise it replaces g with the average of g and x/g.

Worked example · follow the square-root recipe

For $x = 16$ and initial guess $g = 3$, calculate two updates.

1. First compare $3^2 = 9$ with 16, so continue. The next guess is $(3 + 16/3)/2 \approx 4.167$.
2. Now $4.167^2 \approx 17.36$, so update again: $(4.167 + 16/4.167)/2 \approx 4.003$.
3. The next estimate is about 4.000. The procedure approaches $\sqrt{16}$ and stops when its chosen closeness test succeeds. §2.1

Trap: an iteration rule is not complete until it states what counts as close enough and what happens when that test is not met.

Recall check · answer without looking

1. Name the three practical ingredients of an algorithm from the slides.
2. What does the square-root procedure do when its current guess is not close enough?
3. How do finiteness and halting differ?

Answers: see "2.1 Algorithms are finite, executable procedures" under Answers at the end.

2.2 A stored-program computer executes instructions

A stored-program computer can carry out many different procedures because its instructions are data stored in memory. The same machine follows a different algorithm when the stored program changes.

Instructions, control flow, and architecture

A stored-program computer keeps a sequence of instructions in memory. A processor executes them using primitive operations such as arithmetic or logic, simple tests, and moving data. Tests can change which instruction runs next; execution stops when the program says it is done or reaches its stopping condition.

Machine-level view

$$\text{program in memory} \rightarrow \text{processor executes instructions} \rightarrow \text{input becomes output}$$

The slide's architecture diagram groups processor and control logic with registers, memory, input, and output. The diagram is a simplified model; it does not specify a particular modern computer.

The lecture lists six abstract Turing-machine primitives: move right, move left, print a symbol, scan the current symbol, erase it, and halt. Modern programming languages package low-level actions into more convenient operations. The notes emphasize that a computable procedure can be expressed in different general-purpose languages, even though syntax and tools differ.

Memory hook: the algorithm is the recipe; the stored program is that recipe represented as instructions the machine can execute.

Recall check · answer without looking

1. What does the stored-program idea place in computer memory?
2. What are the three broad kinds of primitive instructions listed in the slides?
3. What role can a test play in the flow of a program?

Answers: see "2.2 A stored-program computer executes instructions" under Answers at the end.

2.3 Programs have syntax, static meaning, and behavior

A computer needs instructions that are legal in its language and have a defined interpretation. Separating the kinds of errors helps explain why a program can be rejected, crash, or run while still giving the wrong answer.

Three checks on a program expression

Syntax asks whether the symbols form a legal expression. **Static semantics** asks whether a syntactically legal expression is meaningful under the language's rules, such as whether the operands have compatible types. **Semantics** is the meaning associated with an expression that passes those checks.

Examples from the slides

$$3.2 * 5 \rightarrow \text{syntactically valid and meaningful}$$

$$3 + \text{"hi"} \rightarrow \text{syntactically valid, but a static semantic error}$$

The exact type rules depend on the language. These examples illustrate the distinction, not a specification for every language.

A syntactic error is often caught before execution. Static semantic errors may be caught by a compiler or interpreter. A semantic error can be harder: the program may stop, run forever, or return an answer different from the programmer's intention. A valid program statement has a defined machine meaning, but that meaning may not match what its author intended.

A program can contain definitions and commands. An interpreter evaluates them in a shell, or a compiler translates the program into processor instructions. Commands may be entered directly or saved in a file for the shell to read.

Recall check · answer without looking

1. What kind of error is "3 + 'hi'" meant to illustrate in the slides?
2. How can a program with no syntax or static-semantic errors still fail its purpose?
3. What is one difference between a compiler and an interpreter in the lecture's description?

Answers: see "2.3 Programs have syntax, static meaning, and behavior" under Answers at the end.

2.4 Propositions turn logic into two-valued inputs

Boolean circuits calculate with truth values. Propositional logic gives precise rules for combining statements before those rules are translated into algebra or gates.

Statements and connectives

A **proposition** is a statement that is either true or false. Compound propositions use **NOT**, **AND**, and **OR**. In the notes' example, the umbrella output is true when it is raining or the forecast is bad.

Core connectives

$\neg$A is true exactly when A is false

A $\wedge$ B is true exactly when both A and B are true

A $\vee$ B is true when at least one of A or B is true

The OR in this lecture is inclusive: it is also true when both inputs are true.

For example, let R mean "it is raining" and W mean "the weather forecast is bad". The umbrella decision is $U = W \vee R$. If $W = 0$ and $R = 1$, then $U = 1$.

More complex decisions combine several operations. The lecture gives $U = C' \cdot (W + R)$: take an umbrella when you are not taking the car and either the forecast is bad or it is raining. Parentheses matter because the operations have different precedence.

Recall check · answer without looking

1. What must be true for A $\wedge$ B to be true?
2. Can A $\vee$ B be true when A and B are both true?
3. Write the umbrella rule "raining or bad forecast" with R and W.

Answers: see "2.4 Propositions turn logic into two-valued inputs" under Answers at the end.

2.5 Boolean algebra and truth tables make the rules checkable

Boolean algebra replaces long verbal propositions with symbols and gives every operation a precise meaning. Truth tables then provide a direct way to calculate an expression for every possible input.

Translate propositions into algebra

Boolean values use 1 for TRUE and 0 for FALSE. Variables stand for propositions. The supplied slides write NOT as a prime, OR as $+$, and AND as $\cdot$. Read the notation carefully: here $+$ is logical OR, not ordinary arithmetic addition.

Notation and precedence

$$A' = \text{NOT}\, A$$
$$A \cdot B = A\, \text{AND}\, B$$
$$A + B = A\, \text{OR}\, B$$
$$\text{NOT}\, \text{first} \cdot \text{AND}\, \text{next} \cdot \text{OR}\, \text{last}$$

Use parentheses to make grouping explicit. For example, C' · (W + R) is different from C' · W + R. In C, logical operators are &&, ||, !; bitwise operators are &, |, ~.

Enumerate every input

A truth table lists an expression's output for every assignment of its input variables. With n Boolean variables there are 2^n assignments. Two expressions are equivalent when their output columns match in every row.

A	**B**	**A ∧ B**	**A ∨ B**	**¬A**
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Worked example · check implication

Compare A ⇒ B with A' + B.

1. List inputs (A,B) as (0,0), (0,1), (1,0), (1,1).
2. Evaluate A' + B: 1, 1, 0, 1.
3. By the truth-table definition, A ⇒ B is true when A is false or B is true, so its column is also 1, 1, 0, 1.
4. Matching outputs in every row establishes that A ⇒ B and A' + B are equivalent. §2.5

Trap: three input variables require eight rows, not six. List all assignments before calculating an output column.

Recall check · answer without looking

1. How many rows are needed for five input variables?
2. What does it mean for two Boolean expressions to be equivalent?
3. Which operation has lower precedence: AND or OR?

Answers: see "2.5 Boolean algebra and truth tables make the rules checkable" under Answers at the end.

2.6 Boolean laws simplify expressions

Boolean laws let you transform an expression while preserving its output. Each identity can be checked with a truth table, and the same law can simplify the design of a circuit.

Core identities

Negation, identity, and idempotence

$$(A')' = A$$
$$A \cdot A' = 0$$
$$A + A' = 1$$
$$A \cdot A = A$$
$$A + A = A$$
$$A \cdot 0 = 0$$
$$A \cdot 1 = A$$
$$A + 0 = A$$
$$A + 1 = 1$$

The slides use $\cdot$ for AND and $+$ for OR. A and B may themselves be compound Boolean expressions.

Commutative, associative, and distributive

$$A \cdot B = B \cdot A$$
$$A + B = B + A$$
$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$
$$(A + B) + C = A + (B + C)$$
$$A \cdot (B + C) = A \cdot B + A \cdot C$$
$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

Parentheses keep the grouping visible. The two distributive laws mirror each other with AND and OR exchanged.

De Morgan's rules

Negating a compound expression changes its operation and negates each input: NOT of an OR becomes an AND of the negations, and NOT of an AND becomes an OR of the negations.

De Morgan's laws

$$(A + B)' = A' \cdot B'$$
$$(A \cdot B)' = A' + B'$$

The rules generalize to more than two inputs.

Worked example · simplify a negated expression

Rewrite (A + B)' using De Morgan's law, then evaluate it when A = 0 and B = 1.

1. Apply (A + B)' = A' · B'.
2. Substitute the inputs: $1 \cdot 0 = 0$.
3. The original expression is false because at least one input to the OR was true. §2.6

Recall check · answer without looking

1. State the De Morgan rule for NOT(A OR B).
2. What are the idempotent laws?
3. How can you check a proposed Boolean identity for two variables?

Answers: see "2.6 Boolean laws simplify expressions" under Answers at the end.

2.7 Logic gates implement Boolean functions

A gate is a circuit block whose output follows a Boolean rule. Algebra connects a circuit diagram to its expression and makes it possible to build several operations from a universal gate.

Read and combine gates

AND, OR, and NOT gates represent the corresponding operations. A small circle at a gate output indicates inversion, producing NAND or NOR. Follow a multi-gate circuit from its inputs toward its output, giving each intermediate signal a name.

Inverted gates and exclusive gates

$$\begin{aligned}\mathrm{NAND}(A,B) &= (A \cdot B)' \\ \mathrm{NOR}(A,B) &= (A + B)' \\ \mathrm{XOR}(A,B) &= A' \cdot B + A \cdot B' \\ \mathrm{XNOR}(A,B) &= A' \cdot B' + A \cdot B\end{aligned}$$

XOR is true when exactly one input is true. XNOR is true when both inputs agree.

NAND is a universal gate

The lecture calls a set of functions complete when combinations of those functions can generate any Boolean function. NAND alone is complete, so NOT, AND, and OR can each be assembled from NAND gates.

Worked example · translate a two-stage NAND circuit

The first NAND receives B and C and produces I. A second NAND receives A and I and produces X.

1. Name the intermediate output: I = (B · C)'.
2. Write the final gate: X = (A · I)' = (A · (B · C)')'.
3. Use (P · Q)' = P' + Q': X = A' + ((B · C)')'.
4. Apply double negation: X = A' + B · C. §2.7

Memory hook: bubbles on outputs mean "invert this result"; translate them as a prime before simplifying.

Recall check · answer without looking

1. When is XOR true?
2. How do you express NAND using AND and NOT?
3. What does functional completeness mean for NAND?

Answers: see "2.7 Logic gates implement Boolean functions" under Answers at the end.

2.8 Physical circuits take time to switch

A Boolean truth table describes the stable input-output rule. Real circuits also have timing behavior because their physical components need time to change state.

From ideal logic to a real signal

The final slides contrast an ideal, instantaneous gate with real switching. They identify time delay and saturation as limits on how quickly a physical circuit can change its output. A logic diagram therefore describes function, while a timing diagram is needed to reason about when a change becomes visible.

Two useful models

$$\text{Boolean model} \rightarrow \text{which stable output follows each input?}$$
$$\text{Physical model} \rightarrow \text{how long does the output take to reach its new state?}$$

The supplied slides introduce the distinction but do not provide numerical delay values or a timing-analysis procedure.

Trap: do not claim that a real gate changes output at exactly the same instant as its input. The lecture explicitly notes switching delay.

Recall check · answer without looking

1. What two real-circuit limits are named in the lecture?
2. What question does a truth table answer that a timing diagram does not?
3. Does this lecture supply a numerical propagation delay for the example gate?

Answers: see "2.8 Physical circuits take time to switch" under Answers at the end.

Formula map and symbols

Idea	Rule	Check
Truth-table size	n inputs $\rightarrow 2^n$ rows	Enumerate every assignment once.
Implication	$A \Rightarrow B = A' + B$	False only for A=1, B=0.
Complement of OR	$(A + B)' = A' \cdot B'$	Both inputs must be 0.
Complement of AND	$(A \cdot B)' = A' + B'$	At least one input must be 0.
XOR	$A' \cdot B + A \cdot B'$	Inputs differ.
XNOR	$A' \cdot B' + A \cdot B$	Inputs agree.
Two-stage NAND example	$X = A' + B \cdot C$	Label the intermediate output first.

Check every answer: confirm the operator precedence, list every input assignment, and test the expression against one row that should make it true and one that should make it false.

Derive it cold

Rebuild each result from its definition before opening the worked examples.

1. Run the square-root update twice from $x = 16$ and $g = 3$; show each average. §2.1
2. Explain why the sequence, control flow, and stop rule all matter to a recipe. §2.1
3. Build the truth table for A ⇒ B and show its equivalence to A' + B. §2.5
4. Prove one De Morgan law by comparing its truth-table outputs. §2.6
5. Translate the two-stage NAND circuit into an expression, then simplify it to X = A' + B · C. §2.7

Explain it

Answer each question aloud in your own words, then compare with the model answer.

1. Why is a stopping condition part of an algorithm?
2. Why can the same computer execute different algorithms?
3. Why is "3 + 'hi'" different from an expression with invalid syntax?
4. Why does a truth table with n inputs have 2^n rows?
5. Why do parentheses change C' · (W + R) compared with C' · W + R?
6. Why is NAND called functionally complete?
7. Why is a Boolean truth table not enough to describe a real circuit's timing?

Answers: see "Explain it" under Answers at the end.

Mixed practice

Try all 20 problems before opening the worked answers. They alternate across topics so you must choose the rule, not just repeat the previous one.

1. List the three ingredients of a recipe that make it an algorithm in the practical definition from the lecture.
2. Let A=1 and B=0. Evaluate A ∧ B, A ∨ B, and ¬A.
3. A machine stores instructions and uses tests to change which step runs next. Which computer model from the lecture does this describe?
4. How many rows are needed for a truth table with four Boolean inputs?
5. Classify "3 + 'hi'" using the lecture's distinction between syntax and static semantics.
6. Rewrite (A · B)' using a De Morgan law.
7. For A=1 and B=0, evaluate A ⇒ B and A' + B. What do you observe?
8. For x=16 and current guess g=3, calculate the next square-root guess using (g + x/g)/2. Round to three decimals.
9. Use De Morgan's law to rewrite NOT(A OR B) using primes and ·.
10. State the difference between an ideal Boolean gate model and a physical circuit model.
11. Let A=0, B=1, C=1. Evaluate A' + B · C using the stated precedence.
12. Give one possible behavior of a semantic error even when a program is syntactically valid.
13. For A=1 and B=1, find A XOR B and A XNOR B.
14. Simplify A · A + A using Boolean laws.
15. In C notation, which operator is logical AND: & or &&?
16. A two-stage NAND circuit has I=(B · C)' and X=(A · I)'. Simplify X.
17. After the first square-root update, explain what the algorithm does if the stopping test still fails.
18. Simplify (A + B)' · (A + B) using a complement law.
19. When A=1, B=0, C=1, evaluate X=A' + B · C.
20. What is the maximum number of Boolean functions of three input variables? Treat a Boolean function as one choice of output bit for each truth-table row.

Answers: see "Mixed practice" under Answers at the end.

Exam-style problems

Set a timer for about 20 minutes per problem. Show intermediate steps and identify the rule behind each conclusion.

1 · From a recipe to an algorithm

For this practice variation on the lecture's square-root recipe, make the stopping test explicit. For positive x, start with a positive guess g, test whether $|g^2-x|$ is below a tolerance ε, stop if so, and otherwise set g to $(g+x/g)/2$ and repeat.

1. Write the sequence of steps in plain language, including the control flow and stop condition.
2. For x=16, g=3, and ε=0.01, calculate two successive guesses to three decimals and decide whether the second estimate passes the stopping test.
3. Explain which algorithm properties are demonstrated by the finite instruction list, the tolerance test, and the repeated update. Name one additional condition needed to justify that every run halts.

Answers: see "Exam-style problems" under Answers at the end.

2 · Simplify and check a NAND circuit

A circuit has I=(B·C)' and X=(A·I)'. Assume Boolean inputs.

1. Use Boolean algebra to simplify X.
2. Complete the output values for A,B,C equal to 0,1,1 and 1,1,1.
3. Explain why the circuit can be built using NAND gates alone and why its physical output may not change instantaneously.

Answers: see "Exam-style problems" under Answers at the end.

Self-check

Use day 0 to learn and practise. Tick a skill only in a later session, at least one day afterward, when you can demonstrate it without notes. Listed problems provide evidence.

- ☐ Explain the algorithm requirements and run the square-root update. (1, 8, 17)
- ☐ Describe the stored-program model and instruction classes. (3)
- ☐ Distinguish syntax, static semantics, and semantic errors. (5, 12, 15)
- ☐ Evaluate NOT, AND, and inclusive OR. (2, 7)
- ☐ Determine truth-table size and enumerate inputs. (4, 20)
- ☐ Apply precedence and parentheses correctly. (11, 15)
- ☐ Show A⇒B=A'+B with a complete truth table. (7)
- ☐ Use complement, idempotent, and De Morgan laws. (6, 14, 18)
- ☐ Translate and simplify a NAND circuit. (9, 16, 19)
- ☐ Distinguish XOR from XNOR by input agreement. (13)
- ☐ Explain the difference between Boolean behavior and switching time. (10)
- ☐ Rebuild the listed derivations from definitions.
- ☐ Solve both multi-part problems and justify each step.

Answers

Start from memory

1. It has well-defined executable steps, finite resource use, and halts with an output for its input. §2.1
2. Syntax concerns whether a symbol sequence is grammatically valid; semantics concerns the meaning of a valid expression. Static semantics checks whether a valid-looking expression is meaningful in the language. §2.3
3. $2^3 = 8$ rows, one for each assignment. §2.5
4. Only when A is true and B is false. §2.4
5. Every Boolean function can be built using NAND gates alone. §2.7

2.1 Algorithms are finite, executable procedures

1. Simple steps, control flow that determines when they run, and a rule for stopping. §2.1
2. It averages the guess g with x/g and repeats the test. §2.1
3. Finiteness says the description and each resource use stay finite; halting says the run takes a finite number of steps and terminates. §2.1

2.2 A stored-program computer executes instructions

1. A sequence of instructions that the machine can execute. §2.2
2. Arithmetic and logic, simple tests, and moving data. §2.2
3. It can determine which instruction or branch executes next. §2.2

2.3 Programs have syntax, static meaning, and behavior

1. A static semantic error: the expression's pieces are legal-looking but their combination is not meaningful under the language's rules. §2.3
2. Its behavior may differ from the programmer's intention: it can crash, fail to halt, or return an unexpected answer. §2.3
3. An interpreter executes/evaluates instructions in a shell; a compiler translates a program into machine instructions. The slide gives this as a broad distinction. §2.3

2.4 Propositions turn logic into two-valued inputs

1. Both A and B must be true. §2.4
2. Yes. The lecture uses inclusive OR. §2.4
3. $U = R \vee W$ (or $U = R + W$ in the Boolean notation introduced next). §2.4

2.5 Boolean algebra and truth tables make the rules checkable

1. $2^5 = 32$ rows. §2.5
2. Their outputs match for every possible input assignment. §2.5
3. OR has lower precedence; NOT is highest and AND is in the middle. §2.5

2.6 Boolean laws simplify expressions

1. (A + B)' = A' · B'. §2.6
2. A · A = A and A + A = A. §2.6
3. Make a truth table for both sides and compare the output columns in all four input rows. §2.6

2.7 Logic gates implement Boolean functions

1. When exactly one of its two inputs is true. §2.7
2. (A · B)'. §2.7
3. Any Boolean function can be built from combinations of NAND gates alone. §2.7

2.8 Physical circuits take time to switch

1. Time delay and saturation limit the switching speed. §2.8
2. It gives the stable logical output for each input assignment. §2.8
3. No. The slides make the qualitative point but give no numeric delay value. §2.8

Explain it

1. Without a stopping condition, the procedure may keep repeating and never return an answer. §2.1
2. The stored instructions can change while the hardware remains the same; the machine follows the program it is given. §2.2
3. The second expression is syntactically valid but has a static semantic problem: the operands cannot be combined meaningfully under the illustrated rules. §2.3
4. Each variable has two possible values, so all combinations number 2 multiplied by itself n times. §2.5
5. NOT, AND, and OR have a precedence order; without parentheses the expression groups differently and can produce a different output. §2.5
6. Any Boolean function can be assembled using NAND gates alone. §2.7
7. A truth table describes stable input-output values, while real gates have switching delay and saturation limits. §2.8

Mixed practice

1. A sequence of simple steps, control flow that determines when each step runs, and a way to know when to stop. §2.1
2. A ∧ B=0, A ∨ B=1, and ¬A=0. §2.4
3. A stored-program computer. §2.2
4. 2^4=16 rows. §2.5
5. It is syntactically valid but has a static semantic error in the slide's example. §2.3
6. A' + B'. §2.6
7. Both are 0. Their matching outputs for this row are consistent with the equivalence; check all four rows to prove it. §2.5
8. $(3 + 16/3)/2 = 25/6 \approx 4.167$. §2.1
9. $(A + B)' = A' \cdot B'$. §2.6
10. The Boolean model gives stable outputs for inputs; a physical circuit also takes time to switch and has limits such as saturation. §2.8
11. NOT first gives A'=1; AND next gives B·C=1; OR last gives 1+1=1. §2.5
12. It may crash, run forever, or return an answer different from the programmer's intention. §2.3
13. XOR=0 and XNOR=1 because the inputs agree. §2.7
14. A·A=A, so A·A+A=A+A=A. §2.6
15. && is logical AND; & is bitwise AND. §2.5
16. X=(A·(B·C)')'=A'+B·C. §2.7
17. Replace g by the average of g and x/g, then test the new guess again. §2.1
18. Let Y=A+B. Then Y'·Y=0 by the complement law. §2.6
19. X=0+0·1=0. §2.7
20. There are 8 truth-table rows and two possible output choices per row, so 2^8=256 functions. §2.5

Exam-style problems

1. Set the inputs x, g, and ε. Compute $|g^2-x|$. If it is less than ε, output g and stop; otherwise replace g with (g+x/g)/2 and repeat the test. §2.1
2. First update: (3+16/3)/2≈4.167. Second (using unrounded 25/6): (25/6+16/(25/6))/2≈4.003333, shown as 4.003. Its residual from the unrounded iterate is about 0.02668; using the displayed 4.003 gives about 0.024. Both exceed ε=0.01, so another update is needed. §2.1
3. The finite list is a finite description; its defined arithmetic and comparison steps are executable; the tolerance defines a stopping test. To establish halting for every allowed input, one must show the update eventually makes the residual less than ε (and specify valid inputs such as x>0 and ε>0). §2.1

Exam-style problems

1. Substitute I: X=(A·(B·C)')'. De Morgan gives X=A'+((B·C)')'. Double negation gives X=A'+B·C. §2.6 §2.7
2. For 0,1,1: X=1+1·1=1. For 1,1,1: X=0+1·1=1. §2.5
3. The original circuit already uses NAND gates, and NAND is functionally complete, so any Boolean function can be built from it. Physical switching has delay and saturation limits. §2.7 §2.8

Personal study guide based on the supplied APDR lecture slides, "Algorithms, programming and data representation" and "Boolean algebra and logic," in 02_computers.pdf (slides 1–49, dated 14 April 2024; the Boolean-logic section credits Bernhard Kainz). Written in my own words; no lecture pages or figures are reproduced. The guide's section numbers group the unnumbered slide topics. Not an official FAU publication; check course details against the current notes.